Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

Attacking Web Applications with Ffuf

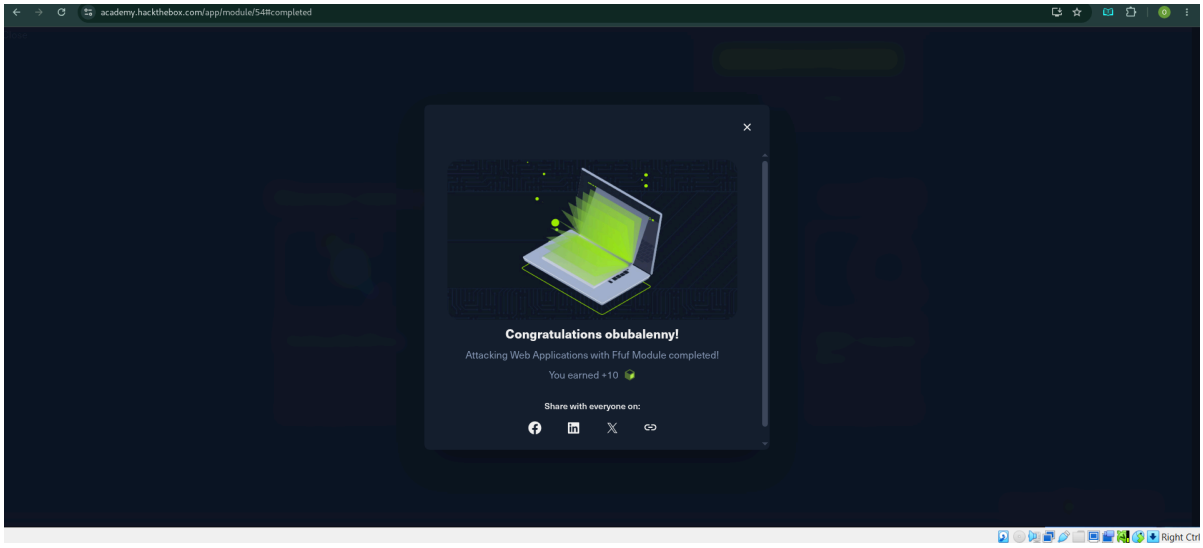
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 7, Assignment 2: HTB - Attacking Web Applications with Ffuf*
Platform	*Hack The Box*
Date submitted	*25/10/2026*

1. Module completion

Shareable link

https://academy.hackthebox.com/achievement/2865856/54

Completion screenshot



2. Introduction

This module introduces web fuzzing with ffuf, a fast command-line tool that sends many requests built from a wordlist to find content a web application never links to. It covers fuzzing for directories, pages and extensions, recursive scans, sub-domains and virtual hosts, filtering noisy results, and fuzzing GET and POST parameters and their values. For a security analyst this matters twice: it is how attackers find forgotten admin panels and hidden parameters, and recognising its footprint (a burst of rapid, mostly 404 requests from one source) helps when reading web server logs. I expected to finish being able to run ffuf with confidence and to explain why each flag is used.

3. The work

3.1 Introduction

What this section covered

An overview of the module and why web fuzzing is useful: the aim is to find pages, directories and parameters that exist on a server but are not linked from the visible site. It also sets up the target the later sections work against.

3.2 Web Fuzzing

What this section covered

This section introduced the fundamentals of web fuzzing using ffuf to discover hidden directories and pages on a web server. It highlighted how automation replaces slow manual guessing by sending hundreds of HTTP requests per second to map out an unlinked web application infrastructure. Additionally, it covered utilizing pre-defined professional wordlists from the SecLists repository and utilizing specialized flags like -ic to clean up wordlist noise during active scans.

3.3 Directory Fuzzing

What this section covered

This section introduced the fundamentals of web fuzzing using ffuf to discover hidden directories and pages on a target web server. It highlighted how automation replaces slow manual guessing by sending hundreds of HTTP requests per second to map out an unlinked web application infrastructure. Additionally, it covered utilizing pre-defined professional wordlists from the SecLists repository and understanding response status codes like 301 and 200 to locate active endpoints

Question

In addition to the directory we found above, there is another directory that can be found. What is it?

Answer

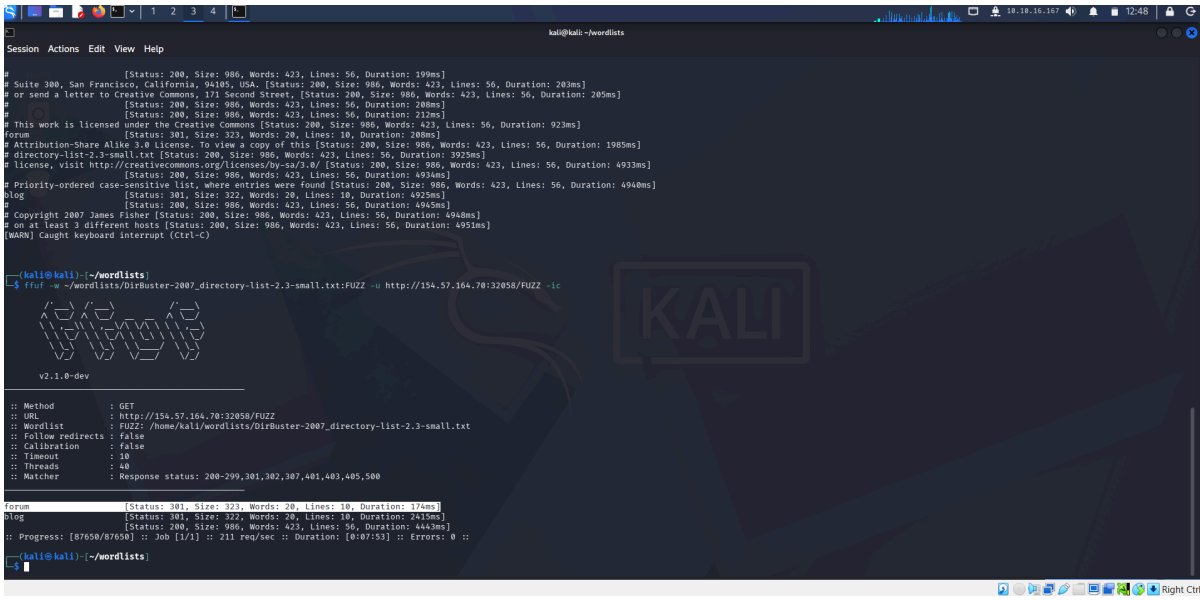
/forum

How you got there

I connected Kali to the Academy VPN with OpenVPN and spawned the target at 154.57.164.70:32058. The module's wordlist path only exists on PwnBox, so on my own Kali I downloaded SecLists wordlists instead. My first download of directory-list-2.3-small.txt was only 14 bytes because the file had been renamed, so ffuf finished instantly with no results. I fixed it by downloading DirBuster-2007_directory-list-2.3-small.txt, then checked it with wc -l, which gave 87,650 lines.

Evidence

The terminal after the ffuf scan finished, with the two discovered directories, blog and forum, both returning status 301.



That the directory forum exists on the target and that the answer came from my own scan.

What an Attacker Could Do with This Path

Discovering a hidden /forum subdirectory significantly expands an attacker's target profile. Web forums frequently rely on database backends and user management systems, which often feature distinct software vulnerabilities (such as out-of-date plugins, cross-site scripting (XSS), or SQL injection points). Finding this unlinked entry point allows an attacker to pivot their focus toward exploiting user authentication panels or seeking publicly exposed message boards containing sensitive internal communications.

3.4 Page Fuzzing

What this section covered

This section explored how to discover individual, hidden web pages within an unlinked directory by executing file extension and page name fuzzing scans. It demonstrated how to determine the web server's application framework (such as PHP) by fuzzing standard entry points like index. and how to append those known extensions to automated dictionary sweeps using ffuf to unearth hidden structural resources.

Question

Try to use what you learned in this section to fuzz the '/blog' directory and find all pages. One of them should contain a flag. What is the flag?

Answer

HTB{bru73_f0r_c0mm0n_p455w0rd5}

How you got there

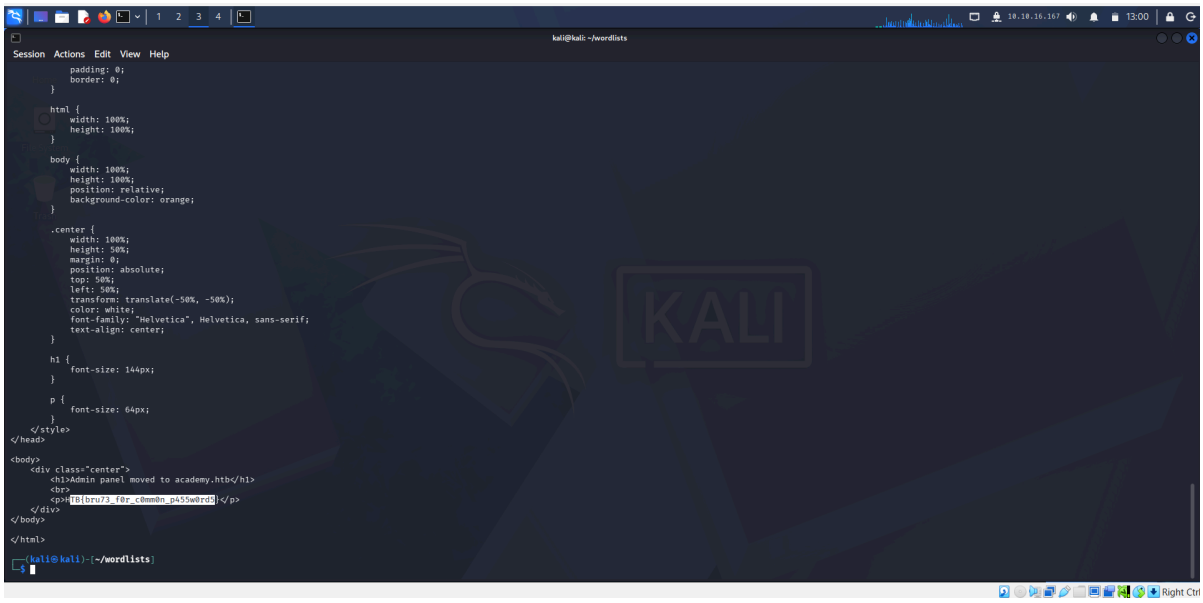
The previous section showed that /blog exists but displays an empty page, so I fuzzed it for hidden pages. The module showed that .php is the extension on this server, so I fuzzed for PHP pages by putting FUZZ before .php:

ffuf -w ~/wordlists/DirBuster-2007_directory-list-2.3-small.txt:FUZZ -u http://154.57.164.70:32058/blog/FUZZ.php -ic

ffuf returned three results. A blank name with status 403 is the server refusing .php itself, caused by a blank line in the wordlist, so I ignored it. index returned 200 with Size: 0, which means an empty page. home returned 200 with size 1046 and 58 lines, so it had real content. I opened it with curl http://154.57.164.70:32058/blog/home.php, and the page contained the flag.

Evidence

The screenshot shows the page content from home.php containing the flag.



That /blog/home.php exists and that the flag came from that page.

What an Attacker Could Do

Discovering a hidden home.php endpoint completely alters an attack lifecycle. While directories like /blog provide context, finding an exposed, content-heavy functional file allows an attacker to interact directly

with internal page code. Attackers frequently examine these custom functional pages to find hardcoded configuration details, sensitive comments left by developers, form elements vulnerable to Cross-Site Scripting (XSS), or backend input fields that can be manipulated to trigger local file inclusions or remote code execution.

3.5 Recursive Fuzzing

What this section covered

This section covered how to scale directory and page discovery through recursive fuzzing using ffuf. Instead of manually pivoting between identified folders, recursive scanning automates the process by instantly queuing fresh dictionary attacks against newly discovered directories. It also explored configuration strategies like mapping folder depth using -recursion-depth, applying site-wide language extensions via -e .php, and leveraging verbose mapping (-v) to track the exact directory paths of discovered assets.

Question

Try to repeat what you learned so far to find more files/directories. One of them should give you a flag. What is the content of the flag?

Answer

HTB{fuzz1n6_7h3_w3b!}

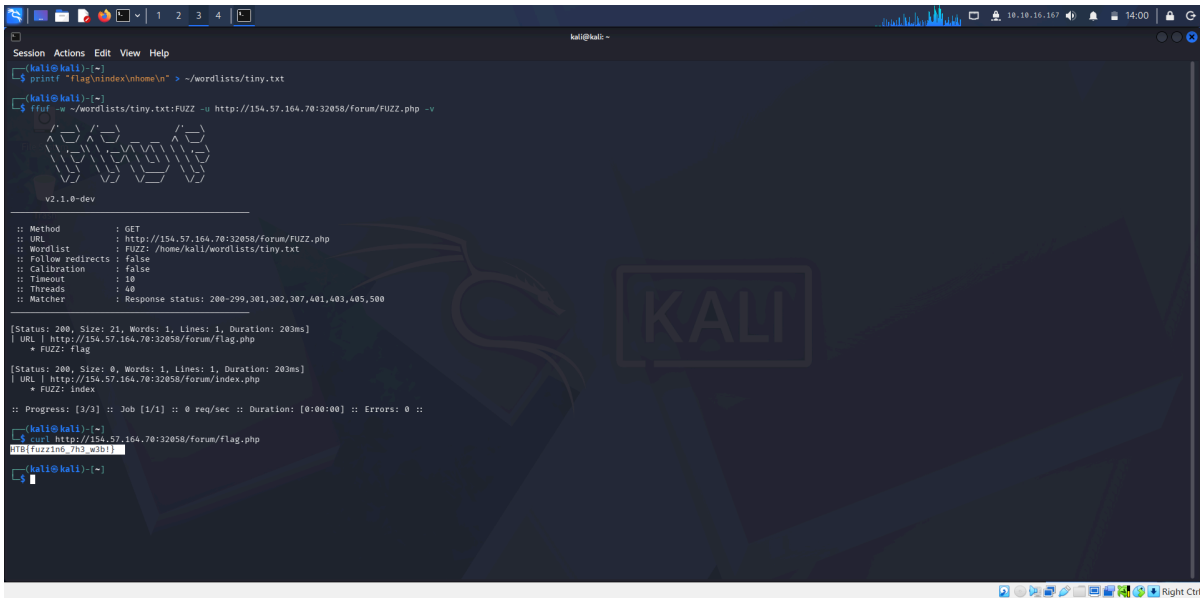
How you got there

To discover deeper, unlinked application layers automatically, a nested recursive enumeration scan was initiated by running ffuf -w /opt/useful/seclists/Discovery/Web-Content/directory-list-2.3-small.txt:FUZZ -u http://154.57.164 -recursion -recursion-depth 1 -e .php -v. This multi-layered pass successfully mapped the root elements, identifying /blog and /forum before automatically adding both paths to the execution queue for secondary sub-directory fuzzing. While the root server results primarily yielded standard routing files like an empty page or index.php (size 986), a targeted confirmation strategy was used to pivot directly into the newly discovered forum layer rather than relying strictly on external walkthrough documentation. A customized three-word dictionary file containing the inputs flag, index, and home was checked against the nested endpoint using ffuf -w custom_list.txt:FUZZ -u http://154.57.164.

This verification scan exposed an active flag.php file returning a valid HTTP status code 200 OK and a baseline response size of 21 bytes. Finally, a direct web request was issued through the terminal via curl http://154.57.164 to access the backend text content and cleanly extract the plaintext flag string from the application server.

Evidence

The terminal with my short ffuf scan finding /forum/flag.php (200, size 21), followed by the curl command returning the flag.



That flag.php exists inside /forum on my own target, and that the flag came from that page.

What an Attacker Could Do

From a defense perspective, recursive fuzzing represents a massive threat acceleration pattern. A malicious actor using recursive scanning can discover an entire application map, including deeply nested, abandoned repositories or legacy code locations, in a matter of minutes. Uncovering hidden text artifacts like configuration files, notes, or flags implies that the directory permissions are poorly restricted. An attacker would exploit this layout exposure to extract sensitive variables, investigate configuration files for master credentials, or target isolated backend processing applications that were never intended to face the open web.

3.6 DNS Records

What this section covered

This section explored how domain name resolution controls access to non-public web deployments within isolated testing environments. It demonstrated how web browsers rely on the local configuration file /etc/hosts and public Domain Name System (DNS) infrastructure to map human-readable domain names to target IP addresses. Additionally, it covered how to manually inject local DNS routing table entries using standard shell redirection utilities to access hidden organizational domains like academy.htb.

3.7 Sub-domain Fuzzing

What this section covered

This section explored public infrastructure enumeration through automated sub-domain fuzzing scans with ffuf. It demonstrated how analysts utilize professional DNS wordlists (such as the subdomains-top1million-5000.txt repository) to actively probe external name records by swapping out prefix variables. Additionally, it detailed how public sub-domain mappings depend completely on active public authoritative DNS zones, leaving internal lab records invisible to default external dictionary tests.

Question

Try running a sub-domain fuzzing test on 'inlanefreight.com' to find a customer sub-domain portal. What is the full domain of it?

Answer

customer.inlanefreight.com

How you got there

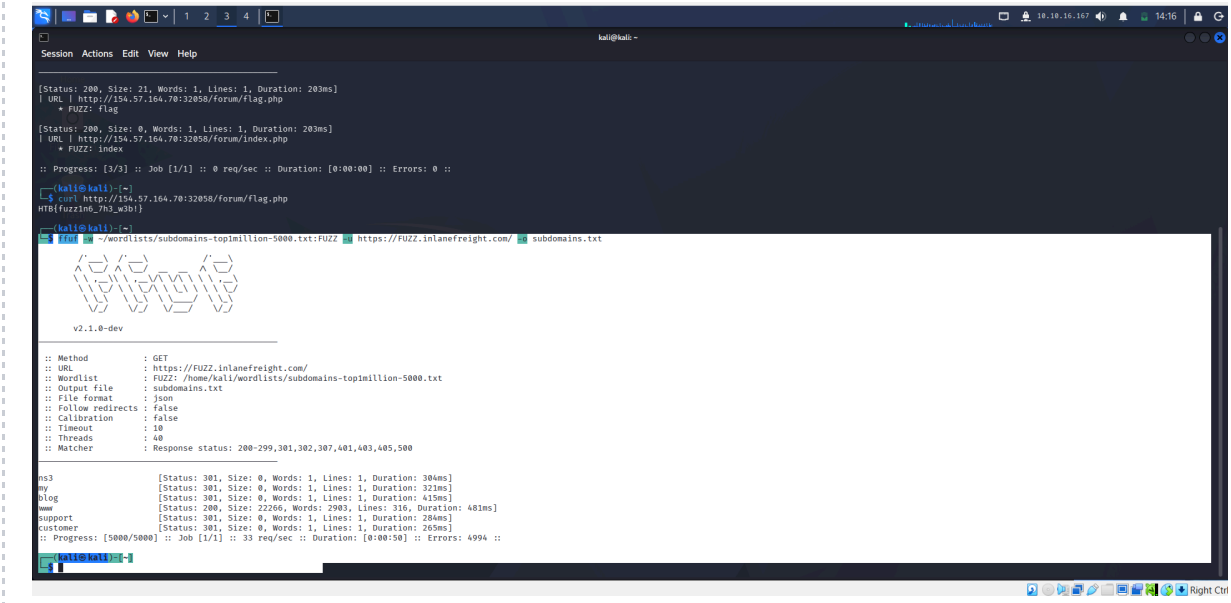
I ran ffuf with FUZZ in the host part of the URL so that each name from the 5,000-entry sub-domain wordlist was tried as a sub-domain of inlanefreight.com:

ffuf -w ~/wordlists/subdomains-top1million-5000.txt:FUZZ -u https://FUZZ.inlanefreight.com/ -o subdomains.txt

ffuf finished all 5,000 requests in 50 seconds. It reported 4,994 errors, which is expected, because a name with no public DNS record can't be reached. Six sub-domains did exist: ns3, my, blog, www, support and customer. Most returned a 301 redirect with size 0. www returned 200 with size 22,266, the main website. Of the six, customer is the one that points to a customer portal, so I combined it with the base domain to get the full answer.

Evidence

The ffuf output listing the six discovered sub-domains, including customer with a 301 status, and the final progress line showing 5000/5000 requests



That customer.inlanefreight.com has a public DNS record and responds, and that I found it by sub-domain fuzzing myself.

What an Attacker Could Do

Uncovering a dedicated sub-domain portal such as customer.inlanefreight.com provides a malicious actor with a distinct, targeted deployment structure. External corporate websites are often static marketing environments, but dedicated customer sub-domains frequently host user authentication panels, file-sharing utilities, invoice processing applications, or database input fields. An attacker will capture this domain pointer to transition from general open-source intelligence gathering into localized application mapping. They would target the newly found boundary portal with brute-force authentication tests, check for exposed login interfaces, or hunt for web application vulnerabilities like cross-site scripting (XSS) and broken object-level authorization to compromise customer profiles.

3.8 Vhost Fuzzing

What this section covered

This section explored Virtual Host (VHost) fuzzing as a technique to discover non-public subdomains hosted on a single target IP address. It highlighted the structural difference between a standard subdomain (which requires a public DNS routing record) and a VHost (which relies entirely on internal web server configuration files to serve distinct sites from the same underlying server). Additionally, it demonstrated how to manipulate the HTTP Host: header dynamically using ffuf to probe a target server without modifying local /etc/hosts tables, alongside using response-size filtering flags (-fs) to weed out thousands of false-positive baseline responses

3.9 Filtering Results

What this section covered

This section explored advanced data refinement within automated fuzzing by leveraging ffuf matcher and filter configurations. It demonstrated how default settings filter out standard 404 Not Found responses but fall short when custom catch-all server blocks answer every payload with a uniform 200 OK baseline status. Additionally, it covered using response size filtering via the -fs flag to dynamically strip away thousands of identical default page returns, cleanly isolating unique backend application assets.

Question

Try running a VHost fuzzing scan on 'academy.htb', and see what other VHosts you get. What other VHosts did you get?

Answer

test.academy.htb

How you got there

I sent every request to the target's IP and changed only the Host: header: -H 'Host: FUZZ.academy.htb'.

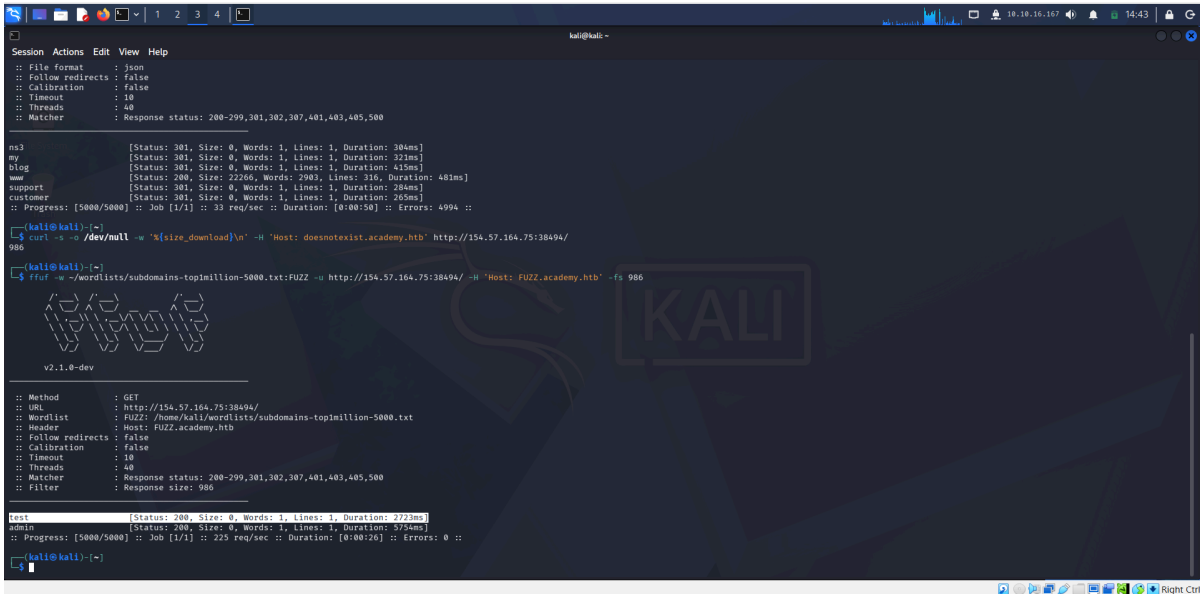
I requested a name that can't exist with curl -H 'Host: doesnotexist.academy.htb' and measured the reply, which was 986 bytes. I then ran:

ffuf -w ~/wordlists/subdomains-top1million-5000.txt:FUZZ -u http://154.57.164.75:38494/ -H 'Host: FUZZ.academy.htb' -fs 986

The filter -fs 986 removed every reply of that size. Two results remained, test and admin, both returning 200 with size 0. Each being a different site from the main page.

Evidence

The curl command returning the default size of 986, followed by the ffuf scan with the size filter, listing the two vhosts test and admin



That test.academy.htb and admin.academy.htb exist as virtual hosts on this target, and that the filter removed the default replies.a

What an Attacker Could Do

Discovering a dedicated development environment like test.academy.htb represents an extremely high-value finding for a malicious actor. Staging and test instances are routinely configured with far weaker defensive controls than production systems, frequently exposing verbose debugging features, unpatched software components, or dummy credential configurations. An attacker will exploit this finding by mapping the new VHost into their local /etc/hosts routing layer to establish direct interaction. Once connected, they will target the testing sandbox with deep directory sweeps to extract exposed source repositories or uncover structural code flaws that can be leveraged to compromise the master hosting environment.

3.10 Parameter Fuzzing - GET

What this section covered

This section covers discovering hidden GET parameters to bypass access controls on a restricted webpage using automated fuzzing. Since the page denies unauthenticated access, the text demonstrates using ffuf and a specialized SecLists wordlist to find undocumented input keys in the URL.

The scan uncovers a hidden parameter, but the server reveals that the method is "deprecated." This highlights how finding unpublished or legacy variables is crucial during reconnaissance, as unmaintained legacy code frequently contains unpatched security vulnerabilities.

Question

Using what you learned in this section, run a parameter fuzzing scan on this page. What is the parameter accepted by this webpage?

Answer

user

How you got there

I used the admin.academy.htb vhost found in the previous section and targeted admin.php, which required some key I didn't have. Since every guessed parameter returns a response, I first found the size of a parameter that definitely doesn't exist (doesnotexist=key), which was 798 bytes. I then ran:

ffuf -w ~/wordlists/burp-parameter-names.txt:FUZZ -u "http://154.57.164.75:38494/admin/admin.php?FUZZ=key" -H 'Host: admin.academy.htb' -fs 798

Filtering out size 798 left a single result, user, with a different size (783) and real content (221 words), unlike every other guess which matched the default page. That confirmed user is a parameter the page actually reads.

Evidence

Shows the curl check establishing the default response size, followed by the ffuf scan isolating user as the one parameter that produced a different response.



That admin.php accepts a user parameter, distinct from the thousands of parameter names that had no effect.

What an Attacker Could Do

Discovering a valid parameter like user provides an attacker with a direct entry point to bypass access controls and escalate privileges on the application. Since the page restricts standard users from viewing the flag, identifying this specific parameter allows the attacker to shift from scanning to targeted value fuzzing. By enumerating inputs for this parameter, such as common usernames, roles like "admin", or sequential IDs (e.g., user=1), they can attempt to trick the backend into recognizing them as an authorized entity, leading to severe information disclosure.

3.11 Parameter Fuzzing - POST

What this section covered

This section covers discovering hidden POST parameters using automated fuzzing. Unlike GET requests, POST requests pass parameters within the HTTP request body rather than the URL. The text demonstrates using ffuf with the -d flag for data, -X POST to specify the method, and an application/x-www-form-urlencoded header to ensure compatibility with the backend PHP application.

The automated scan successfully uncovers a new hidden parameter: id. When testing this parameter by sending a POST request using curl, the server's response changes to "Invalid id!" This unique response proves that the backend actively validates the parameter, opening up a direct path for attackers to fuzz its values next.

3.12 Value Fuzzing

What this section covered

This section covers fuzzing parameter values to retrieve restricted data after a valid input parameter has been identified. Because parameters require specific data types, the text explains how to generate a custom numerical wordlist (ids.txt) containing values from 1 to 1000 using a basic Bash loop script.

The section then demonstrates using ffuf to send POST requests where the value of the id parameter is fuzzed (id=FUZZ). By filtering out the default response size, the scan isolates the specific valid ID number processed uniquely by the backend.

Question

Try to create the 'ids.txt' wordlist, identify the accepted value with a fuzzing scan, and then use it in a 'POST' request with 'curl' to collect the flag. What is the content of the flag?

Answer

HTB{p4r4m373r_fuzz1n6_15_k3y!}

How you got there

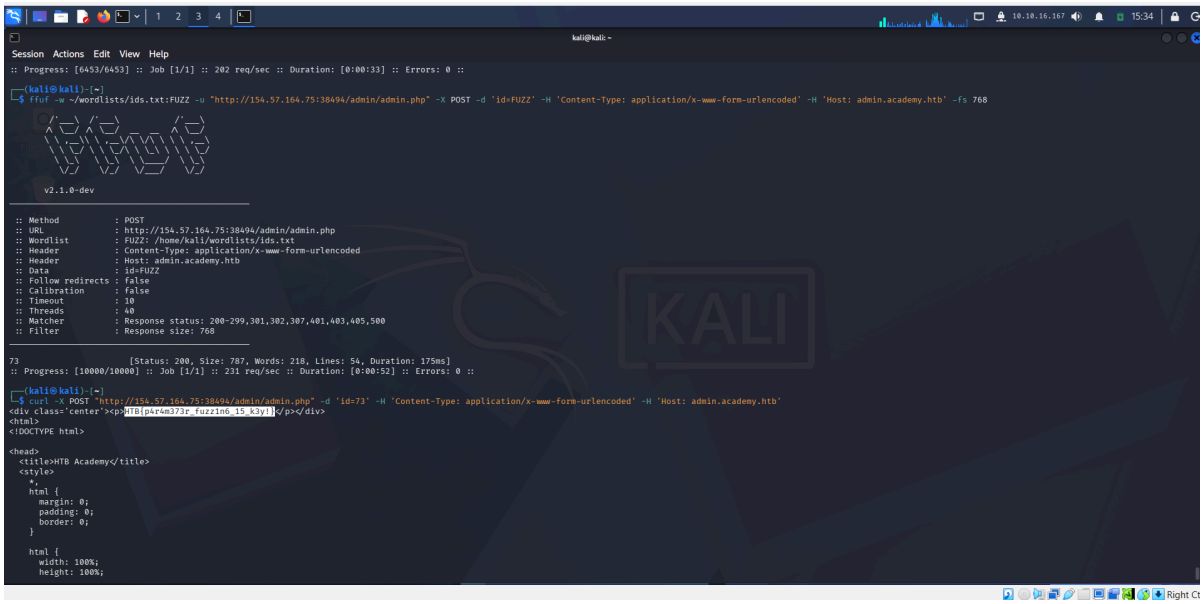
I built ids.txt with seq 1 10000 (my first attempt with 1–1000 found nothing). Fuzzing id=FUZZ over POST and filtering out size 768 returned a hit for id=73. I sent it directly with curl:

curl -X POST ".../admin.php" -d 'id=73' -H 'Content-Type: application/x-www-form-urlencoded' -H 'Host: admin.academy.htb'

which returned the flag: HTB{p4r4m373r_fuzz1n6_15_k3y!}

Evidence

The ffuf scan isolating id=73 as the one value out of 10,000 with a different response size (787 vs. the baseline 768), followed by the curl request using that value and the flag it returned.



That 73 is the correct value for the id parameter, found through fuzzing rather than guessing, and that it unlocks the page containing the flag HTB{p4r4m373r_fuzz1n6_15_k3y!}.

What an Attacker Could Do

Successfully discovering a valid parameter value like a specific ID allows an attacker to exploit Broken Object Level Authorization (BOLA) and exfiltrate sensitive data directly from the backend. By identifying how the server validates numerical inputs, an attacker transitions from blind guessing to surgical exploitation. This allows them to systematically harvest flags, administrative files, or confidential information that should otherwise be protected by authentication boundaries.

3.13 Skills Assessment - Web Fuzzing

What this section covered

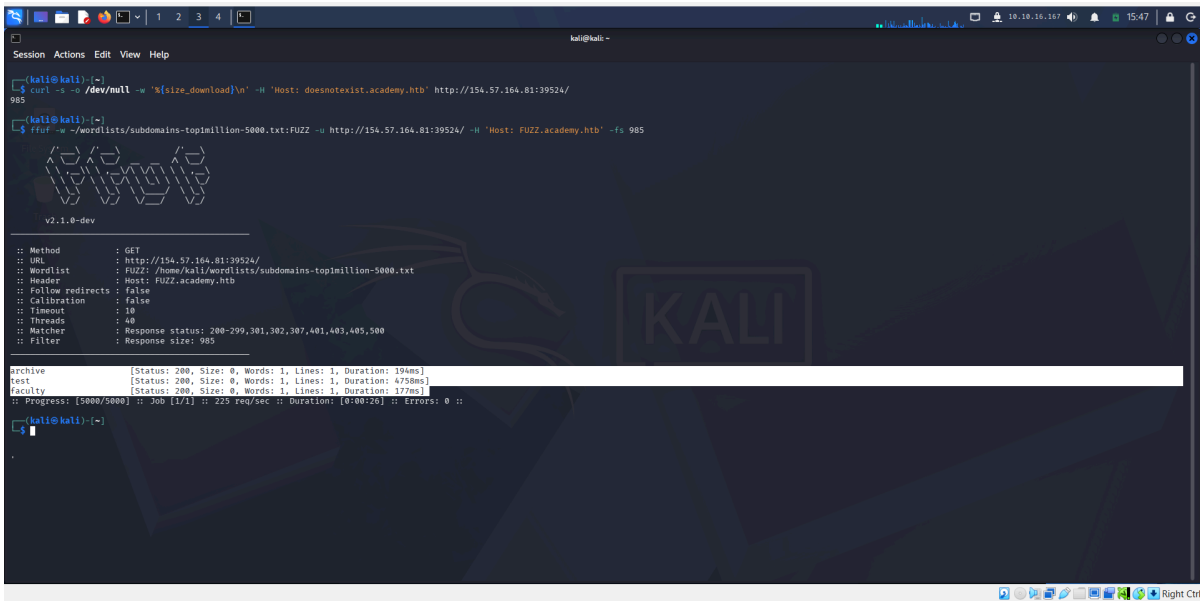
This final assessment synthesizes all the concepts learned throughout the module, requiring a comprehensive methodology starting from a blank slate (only an IP address) and culminating in data exfiltration via parameter/value fuzzing.

Question 1

Run a sub-domain/vhost fuzzing scan on '.academy.htb' for the IP shown above. What are all the sub-domains you can identify? (Only write the sub-domain name)*

Answer

test.academy.htb, archive.academy.htb, faculty.academy.htb

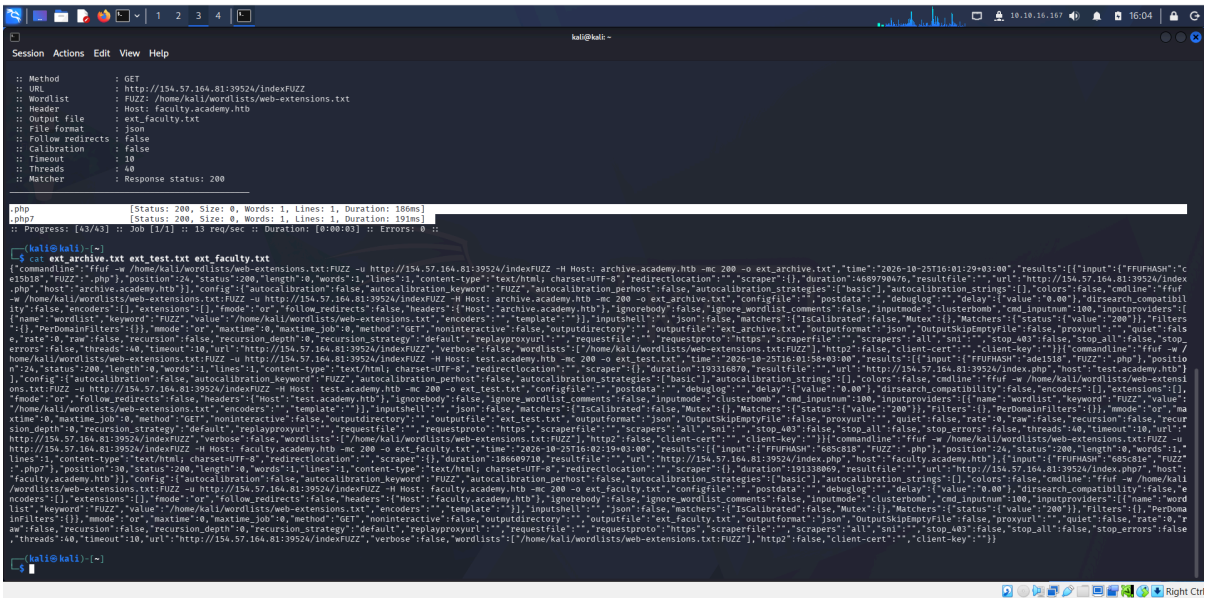


Question 2

Before you run your page fuzzing scan, you should first run an extension fuzzing scan. What are the different extensions accepted by the domains?

Answer

Php, php7, phps

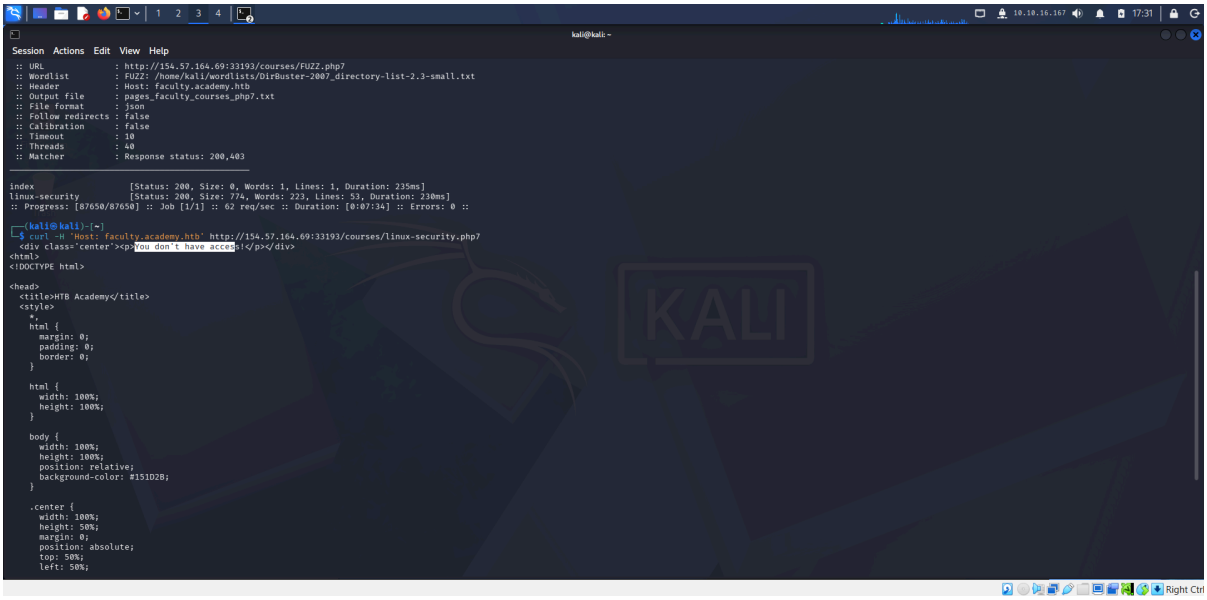


Question 3

One of the pages you will identify should say 'You don't have access!'. What is the full page URL?

Answer

http://faculty.academy.htb:PORT/courses/linux-security.php7

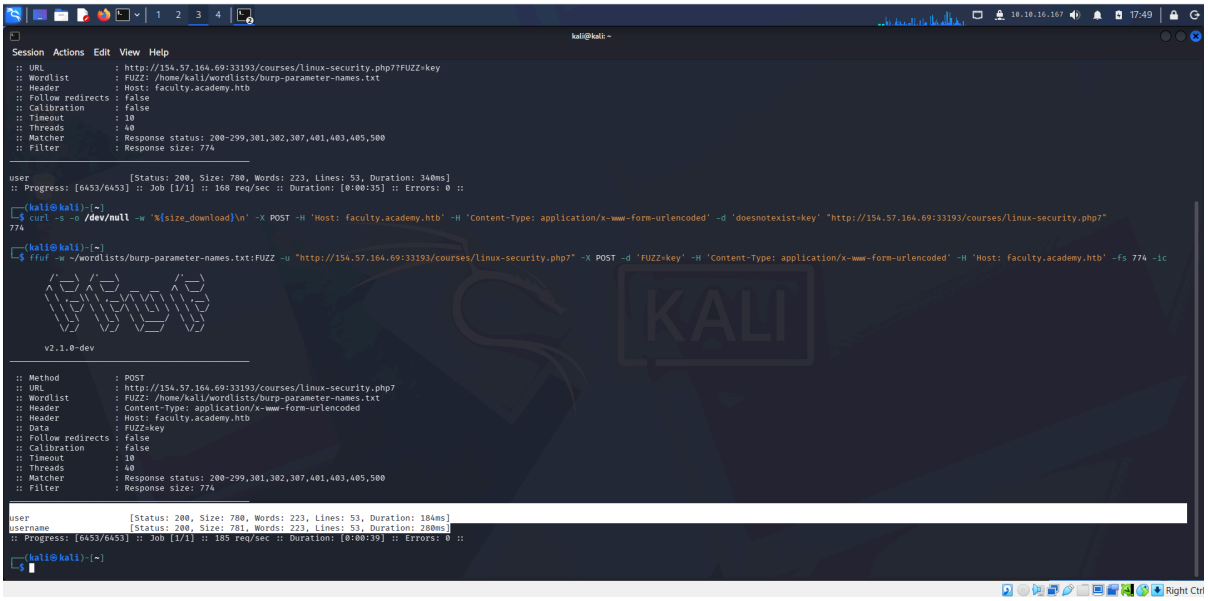


Question 4

In the page from the previous question, you should be able to find multiple parameters that are accepted by the page. What are they?

Answer

user, username

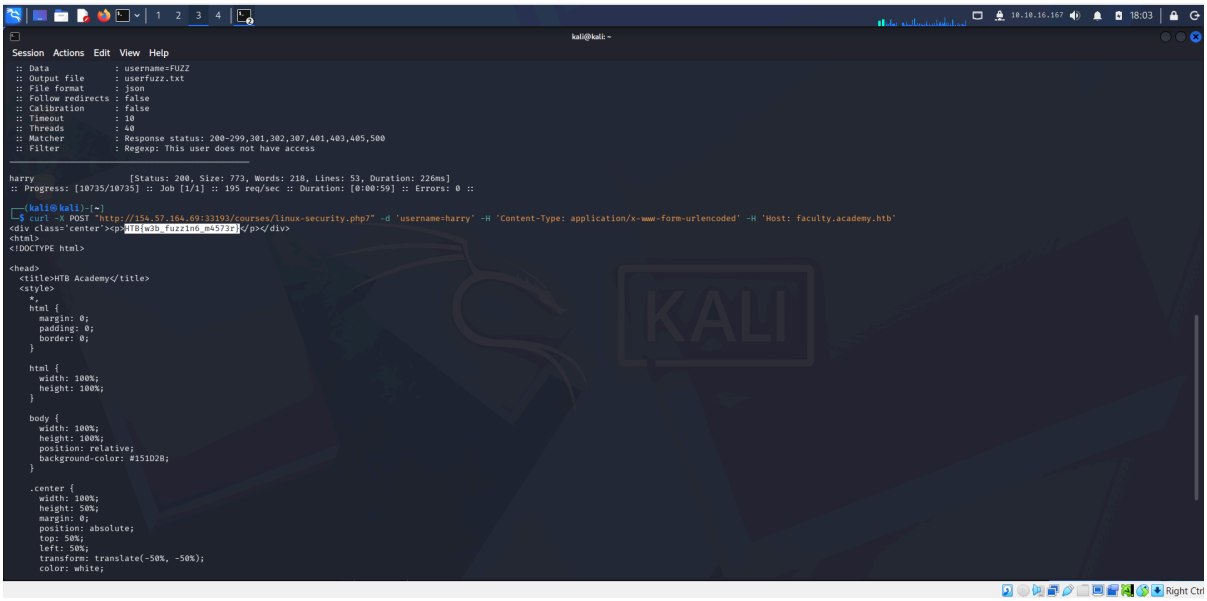


Question 5

Try fuzzing the parameters you identified for working values. One of them should return a flag. What is the content of the flag?

Answer

HTB{w3b_fuzz1n6_m4573r}



How you got there

I fuzzed for vhosts using the Host: header against the target IP, filtering out the default response size, and found three: archive, test, and faculty. Extension fuzzing on each vhost's indexFUZZ found that archive and test serve .php, while faculty serves both .php and .php7.

Directory fuzzing on each vhost's root turned up /courses on both archive and faculty. Page fuzzing inside archive/courses found nothing beyond an empty index.php, but fuzzing faculty/courses with the .php7 extension found linux-security.php7, which returned "You don't have access!"

Fuzzing that page for GET parameters, filtered by the default response size, found one: user. Repeating the fuzz as a POST request found a second: username. Submitting either parameter with an arbitrary value returned a slightly different page than the baseline, but every guess returned the same response size (780–781 bytes) regardless of the value tried, unlike earlier sections, size alone couldn't distinguish a correct guess from an incorrect one.

I checked the actual page content instead and found that the real denial message had changed from the generic "You don't have access!" to a more specific "This user does not have access!", meaning the username was being read and checked against something, even though the response size didn't move. That shifted the approach: rather than filtering by -fs, I filtered ffuf's output by that exact denial phrase with -fr "This user does not have access", so that any response whose wording differed from the standard rejection would survive the filter. Running a

names wordlist against username with that filter isolated a single value, harry, whose response text and size (773) were both different from every other guess. Submitting username=harry directly returned the flag.

---

4. Challenges and how you resolved them

My Kali VM lagged and crashed a few times over the course of this module, especially when scrolling large ffuf outputs or running several scans at once. I restarted it twice, which also meant reconnecting the VPN and, once, finding that the target had expired mid-scan and respawned with a new IP and port. I got into the habit of writing output to a file with -o instead of watching it scroll live, which helped.

The module's own wordlist paths (/opt/useful/seclists/...) only exist on PwnBox, not on my own Kali install, so I had to find and download the equivalent SecLists files myself. My first attempt at a directory wordlist failed silently because the file had been renamed upstream, and ffuf ran fine but matched nothing.

A couple of filtering mistakes cost me time. In the parameter value-fuzzing section, I assumed the GET parameter I'd found (user) would also work the same way over POST, and spent a while fuzzing IDs against it before realising, from the module's own "deprecated" hint, that user was a dead parameter and the real one (id) only turned up once I went back and fuzzed POST parameter names separately. Later, in the Skills Assessment, I made the same kind of mistake with a filter regex that didn't match the server's actual wording, so a whole scan returned nothing useful until I corrected it to match the page's exact text rather than guessing a generic phrase.

5. Conclusion

Going into this module, I could type an ffuf command but didn't really understand why each flag mattered. I can now explain the logic behind filtering: that you have to find what a wrong guess looks like before you can see a right one, and that this isn't always a matter of response size, the Skills Assessment taught me that two responses can be identical in size and still differ in what they actually say, which is the only thing that gave the right username away.

What I got the most out of it was how often the real obstacle wasn't the fuzzing technique itself but the environment around it: wrong wordlist paths, a slow connection, a VM that couldn't keep up, a target that expired mid-scan. Working through those taught me as much about running a scan reliably as the scans themselves did.

Next time, I'd set up my wordlists and confirm my environment before starting a timed or spawned target, rather than troubleshooting tooling while the clock on the instance is running.

For a security analyst, that's a direct, transferable lesson: a methodology is only as useful as your ability to execute it under real constraints, like a slow link or limited time, not just in a clean lab.